Precision as a software choice: FP64, double-double, triple-double and beyond from the INT8 tensor cores of a laptop GPU

Shaban Ejupi

26 October 2026

Abstract

Consumer and laptop GPUs execute FP64 at 1/64 of their FP32 rate, while their INT8 tensor cores are several hundred times faster than their FP64 units. On a laptop Blackwell GPU (RTX PRO 3000, sm_120) we measure 232 TOPS of INT8 `mma.sync` against 0.449 TFLOPS of native FP64, a ratio of 517×. Building on the Ozaki scheme II—exact integer products of residues and reconstruction by the Chinese remainder theorem—we implement, in CUDA C and inline PTX, matrix multiplication at FP64 (5.56 TFLOPS, 12.4× the native peak and more accurate), double-double (∼106 bits, 1.76 TFLOPS), triple-double (∼159 bits, 1.58 TFLOPS) and arbitrary precision (212, 318 and 424 bits) from the same INT8 units. To our knowledge no GEMM beyond FP64 had been built this way. The higher precisions pay off in a real task: solving $Ax = b$ with $\kappa(A) = 10^{20}$ in double-double takes 0.855 s at $n = 8192$, 499× sooner than a `__float128` LU on all 24 CPU threads of the same laptop and with an error as small; at $\kappa = 10^{36}$, where FP64 and double-double have no correct digit, a triple-double LU on the GPU is 32× faster than the same arithmetic on 24 CPU threads and 145× faster than MPFR, and is the most accurate of the three. Counting CPU and GPU power, a double-double solve uses ∼270× less energy than the quad-precision CPU solve. Leaving 8-bit moduli for primes below 2^{16} and 2^{23} split into bytes, an *exact* product of integer matrices with entries of 1 015 to 131 063 bits costs work linear in the bits, is bit-exact against GMP, and runs 10–19× faster than FLINT's multimodular `fmpz_mat_mul` on 24 CPU threads, with 15–18× less energy. Turning the problem around—int8 in, residues out—the exact determinant of an int8 matrix comes 224× sooner than FLINT at $n = 4096$ and 355× at $n = 8192$ (98 139 bits), with a proved bound on $|\det A|$ tight to one bit, and $n = 16\,384$ (204 480 bits) takes 66 s, $n = 32\,768$ (425 348 bits) 594 s within the 12 GB of the GPU. The laptop is power-bound (∼95 W) under these kernels, which shapes what can and cannot be gained.

Përmbledhje (shqip). Kartelat grafike të laptopëve e ekzekutojnë FP64 me 1/64 e shpejtësisë së FP32, ndërsa tensor core-t INT8 janë qindra herë më të shpejtë. Mbi një GPU Blackwell laptopi matëm 232 TOPS INT8 kundrejt 0,449 TFLOPS FP64 (517×). Me skemën Ozaki II (prodhime të sakta të mbetjeve dhe teorema kineze e mbetjeve) ndërtuam shumëzim matricash me saktësi FP64, double-double, triple-double dhe arbitrare (deri 424 bit) nga të njëjtat njësi INT8. Te një punë reale—$Ax = b$ me $\kappa = 10^{20}$—zgjidhja double-double mbaron 499× më shpejt se LU `__float128` mbi 24 fije CPU, me gabim po aq të vogël, dhe me ∼270× më pak energji. Me prime nën 2^{16} e 2^{23} të ndara në bajte, shumëzimi i *saktë* i matricave me numra të plotë 1 015–131 063 bitësh del bit për bit i saktë, 10–19× më shpejt se FLINT dhe me 15–18× më pak energji. Përcaktori i saktë i një matrice int8 del 224× më shpejt se FLINT te $n = 4096$ dhe 355× te $n = 8192$, me një kufi të provuar të ngushtë brenda një biti; te $n = 32\,768$ (425 348 bit) mbaron për 594 s brenda 12 GB të GPU-së.

Table 1: Peak throughput of the laptop GPU (`bench/peak.cu`).

Unit	Throughput
FP64 FMA, native	0.449 TFLOPS
FP32 FMA, native	28.2 TFLOPS
INT8 `mma.sync.m16n8k32` (PTX)	232 TOPS
INT8 / FP64	**517×**

1 Introduction

The FP64 units of consumer and laptop GPUs are a small fraction of the chip: on our laptop Blackwell part a peak kernel reaches 0.449 TFLOPS of FP64 against 28.2 TFLOPS of FP32 and 232 TOPS of INT8 tensor-core throughput (Table 1). Ozaki, Uchino and Imamura showed that FP64 matrix multiplication can be emulated on INT8 tensor cores (Ozaki scheme II): each input is scaled to integers, reduced modulo a set of pairwise coprime moduli, the residue matrices are multiplied exactly in INT8 with INT32 accumulation, and the integer product is reconstructed by the Chinese remainder theorem (CRT) before being scaled back [1]. FP64 by this route is not new; it is our baseline.

What we add is the observation, made concrete and measured, that the same machinery does not stop at FP64. The precision is set by the number of moduli and by how many floating-point words each input and output carries; the expensive part, the INT8 GEMMs, runs at the same speed whatever the precision. This turns precision into a software choice on hardware that has no wide floating-point unit at all. We build and measure:

- FP64 GEMM at 16 moduli: 5.56 TFLOPS, 12.4× the native FP64 peak, with smaller error than native FP64;
- double-double (∼106-bit) GEMM at 32 moduli: 1.76 TFLOPS, 3.9× the native FP64 peak with twice the bits;
- triple-double (∼159-bit) GEMM at 49 moduli—after which every prime below 256 divides one of them, so no further 8-bit modulus can join—at 1.58 TFLOPS;
- arbitrary precision beyond that from balanced 160-bit slices: 212, 318 and 424 bits;
- dense linear solves at condition numbers where FP64 and double-double fail, all on the GPU.

Everything runs on one laptop, and all competitors were measured on the same machine.

2 Method

Ozaki scheme II. For $C = AB$ the rows of A and columns of B are scaled by powers of two to integers A', B' whose product cannot exceed $M/2$ in magnitude, where $M = \prod_\ell m_\ell$ is the product of L pairwise coprime moduli $m_\ell \le 256$. For each ℓ the residues $A' \bmod m_\ell$ and $B' \bmod m_\ell$ fit in INT8, their product is computed exactly by the tensor cores with INT32 accumulation, and reduced mod m_ℓ. The CRT then gives $A'B'$ exactly as a signed integer of up to $\log_2 M$ bits, which is scaled back. The precision of the result is limited only by how many bits of each input survive the scaling, i.e. by $\log_2 M$.

Wider inputs and outputs. For double-double and triple-double inputs, each of the two or three parts is split exactly into 53-bit integer pieces before the residues are formed; like pieces are

summed and cut once into 16-bit chunks, so the residue of a wide integer costs 32-bit multiply-adds per modulus instead of 64-bit reductions (triple-double residues 57 → 19 ms at $n = 4096$). On the output side the integer is never converted through floating-point arithmetic: the CRT is evaluated as a direct sum $x = \sum_\ell r_\ell W_\ell - qM$ over 32-bit limbs on the integer units, with q from 40-bit fixed-point fractions, and the result is peeled off the integer as two or three doubles (round down, round down, round to nearest, each exact from the integer), then normalized. This replaced a Garner reconstruction followed by a double-double Horner evaluation, which took 41.5 of 47 ms of a 4096^2 double-double GEMM because FP64 runs at 1/64 rate on this chip; the direct sum costs 0.68 ns per output element instead of 2.8 and is nearly independent of K. It was checked on the host against 10^5 random 200-bit integers with no error.

Moduli. FP64 uses 16 moduli, double-double 32, triple-double 49 (the 32 plus the 17 primes $103 \ldots 29$ that are not yet factors; $M \approx 2^{342}$). With these, every prime below 256 divides one of the moduli, so no further 8-bit modulus coprime to all of them exists and M cannot grow within one GEMM. Beyond ∼160-bit inputs we cut each input into balanced 160-bit slices and run one 49-modulus GEMM per slice pair that reaches the result, accumulating exactly in wide integers and peeling Q doubles.

3 Accuracy

Errors are the maximum componentwise $|C - C_{\text{ref}}|/(|A||B|)$ in units of the target precision's unit roundoff, against higher-precision CPU references (80-bit for FP64, `__float128` for double-double, an exact 600-bit MPFR sum for triple-double, an exact 2400-bit sum beyond). Inputs have magnitudes spread over $2^{\pm 0}$ or $2^{\pm 8}$.

Table 2: Error in units of 2^{-53} (FP64, $M = N = 256$) and 2^{-106} (double-double, $M = N = 128$).

K	magnitudes	native FP64	14 moduli	**16 moduli**
512	$2^{\pm 0}$	4.71	10.9	**0.99**
512	$2^{\pm 8}$	24.1	395	**3.1**
4096	$2^{\pm 0}$	4.15	10.5	**0.25**
4096	$2^{\pm 8}$	25.1	203	**1.87**
K	magnitudes	24 moduli	28 moduli	**32 moduli**
256	$2^{\pm 0}$	$6.2 \cdot 10^5$	27.1	**1.09**
256	$2^{\pm 8}$	$2.0 \cdot 10^7$	1160	**46.9**
1024	$2^{\pm 0}$	$4.7 \cdot 10^5$	31	**0.49**
1024	$2^{\pm 8}$	$1.1 \cdot 10^7$	713	**12.4**

Table 3: Triple-double error in units of 2^{-159}, $M = N = 128$, against an exact MPFR sum.

K	magnitudes	41 moduli	45 moduli	**49 moduli**
256	$2^{\pm 0}$	$1.7 \cdot 10^4$	5.2	**0.24**
256	$2^{\pm 8}$	$1.2 \cdot 10^6$	343	**10.7**
1024	$2^{\pm 0}$	$1.8 \cdot 10^4$	4.6	**0.13**
1024	$2^{\pm 8}$	$4.3 \cdot 10^5$	110	**2.1**

Beyond triple-double, against an exact 2400-bit sum at $K = 256$: 0.13 ulp at 212 bits, 0.25 at 318 bits, 0.08 at 424 bits.

4 Speed

Table 4: GEMM speed on the laptop. CPU numbers with the CPU otherwise idle, measured at $n = 256/512$ and scaled by n^3.

bits	n	GPU time	GPU TFLOPS	best CPU code, 24 threads	MPFR, 24 threads
53 (FP64)	4096	—	5.56 (12.4× native)	—	—
106 (dd)	4096	—	1.76	—	—
159 (td)	4096	87 ms	1.58	td AVX ∼10.7 s (123×)	∼208 s (2390×)
212	2048	36 ms	0.47	QD `qd_real` ∼16.2 s (445×)	∼35 s (960×)
318	2048	52 ms	0.33	—	∼37 s (713×)
424	2048	80 ms	0.22	—	∼41 s (514×)

At 16 moduli the time of a 4096^3 FP64 GEMM splits into scales 1.0 ms, residues 3.0 ms, the 16 INT8 GEMMs 16.8 ms and the CRT 3.6 ms. The INT8 GEMM reaches 157 TOPS, 67% of the measured peak, with a three-stage `cp.async` pipeline; a faster kernel did not move end-to-end times, because with real data the laptop is power-bound (Section 8).

5 A real task: ill-conditioned linear systems

We solve $Ax = b$ with $A = Q_1 \operatorname{diag}(\sigma) Q_2$, $Q_{1,2}$ orthogonal (a DST-I, the second permuted and sign-flipped), σ spaced from 1 to 10^{-20} or 10^{-36}; such matrices cannot be fixed by scaling. $b = Ax_{\text{true}}$ is summed exactly. The error is $\max|x - x_{\text{ref}}| / \max|x_{\text{ref}}|$.

Double-double, $\kappa = 10^{20}$. κu is 10^4 for FP64, so FP64 LU has no correct digit and textbook iterative refinement diverges (it needs $\kappa u < 1$; at $\kappa = 10^{12}$ the same refinement converges to $3 \cdot 10^{-20}$, so the baseline is right). The GPU solver keeps everything on the device: a recursive panel with cooperative 32-column leaves (a thread per row holding the row in registers, rows renamed instead of swapped, one grid synchronization per column), halves joined by double-double GEMMs on the tensor cores, L_{11}^{-1} assembled from the halves' inverses, left-looking solves with inverted diagonal blocks, and one refinement step in double-double.

Table 5: Double-double LU on the GPU vs `__float128` LU on 24 CPU threads, $\kappa = 10^{20}$, nb = 1024.

n	dd LU, GPU	`__float128` LU	speedup	error vs x_{true} (ours / quad)
2048	0.113 s	7.65 s	67×	$1.6 \cdot 10^{-11}$ / $2.4 \cdot 10^{-11}$
4096	0.283 s	55.6 s	∼ 197×	—
6144	0.572 s	200.8 s	351×	$3.4 \cdot 10^{-10}$ / $2.9 \cdot 10^{-10}$
8192	0.956 s	476.9 s	**499**×	$2.1 \cdot 10^{-11}$ / $2.3 \cdot 10^{-11}$

The 0.956 s at $n = 8192$ was measured in the same run as the quad reference, after minutes of all-core load; alone on a cool GPU the solve takes 0.855 s. Of those, the panel takes 0.45 s (its leaves 0.22 s, bound by the FP64 units at ∼10 GFLOPS per SM), the trailing GEMMs 0.15 s, solves and refinement 0.09 s, U_{12} 0.04 s.

Triple-double, $\kappa = 10^{36}$. Here κu is 10^{20} for FP64 and 10^4 for double-double: neither has a correct digit. The same solver structure in three parts per number, with 16-column leaves:

At $n \le 1024$ a well-vectorized triple-double LU on 24 cores keeps up; the gap opens with n as the work moves into the tensor-core GEMMs. At $n = 4096$ the panels take 0.38 s of the 0.563 s (leaves 0.14 s): the panel, not the GEMM, is now the limit.

Table 6: Triple-double LU, $\kappa = 10^{36}$; GPU speedups in parentheses; CPU otherwise idle.

n	**td LU, GPU**	td LU, 24 CPU threads	MPFR LU, 160 bits	FP64 LU
1024	**0.082 s**, $1.3 \cdot 10^{-14}$	0.10 s, $3.6 \cdot 10^{-14}$ (1.2×)	1.32 s, $3.3 \cdot 10^{-14}$ (16×)	128
2048	**0.212 s**, $2.1 \cdot 10^{-14}$	1.04 s, $7.6 \cdot 10^{-14}$ (5×)	10.2 s, $5.6 \cdot 10^{-14}$ (48×)	103
4096	**0.563 s**, $2.2 \cdot 10^{-14}$	18.0 s, $1.2 \cdot 10^{-13}$ (32×)	81.9 s, $9.6 \cdot 10^{-14}$ (145×)	86

6 Thousands of bits, exactly: linear in the bits

The 49 moduli ≤ 256 cap one Ozaki-II product at ∼160-bit inputs, and slicing beyond that costs a number of GEMMs that grows with the square of the slices. For wide integers we leave 8-bit moduli: the moduli are primes $p < 65\,280$, and each balanced residue ($|r| < p/2$) is split into two signed bytes, $r = 256h + \ell$. Then $AB \bmod p$ is four exact INT8 GEMMs (hh, $h\ell$, ℓh, $\ell\ell$) recombined in 64-bit integers, so *the GEMM work grows linearly with the number of bits.* Above ∼60 000 bits of the CRT modulus the residues take three bytes and the primes run up to 2^{23} (nine products per prime, ∼23 bits each). The two conversions are INT8 GEMMs as well, as conversions to and from a residue number system are matrix products: the residues are the balanced byte digits of each entry times the bytes of $2^{8j} \bmod p$; for the CRT, $t_\ell = r_\ell\,(M/p_\ell)^{-1} \bmod p_\ell$ (bytes) times the bytes of M/p_ℓ gives the digits of $x+qM$, with $q = \mathrm{round}(\sum_\ell t_\ell/p_\ell)$ from doubles, and qM is taken off while the digits are carried. Everything runs on the tensor cores (`src/ozaki_big.cuh`, `bench/big_test.cu`).

Every product is exact: 256 sampled elements per run agree bit for bit with GMP. The baselines compute the same exact product on all 24 CPU threads: GMP inner products (`mpz_addmul`), and FLINT's `fmpz_mat_mul`, which is itself multimodular with BLAS-style kernels and is the strongest baseline we know of.

entry bits	n	primes (bytes)	GPU	GMP	FLINT	GPU vs FLINT
1 015	1024	192 (2)	71 ms	23.0 s	1.31 s	18×
4 087	512	576 (2)	62 ms	16.4 s	1.07 s	17×
16 375	256	2 112 (2)	90 ms	14.0 s	1.69 s	19×
4 087	1024	640 (2)	341 ms	125 s	6.10 s	18×
16 375	512	2 112 (2)	363 ms	104 s	6.33 s	17×
32 759	256	4 352 (2)	316 ms	36.1 s	4.56 s	14×
65 527	128	5 760 (3)	313 ms	10.6 s	3.45 s	11×
131 063	128	11 520 (3)	983 ms	29.6 s	9.76 s	10×

GMP times are measured on a corner of the outputs (96^2, or 48^2 above 8 000 bits) and scaled; FLINT runs the full product. The largest entries have ∼39 450 decimal digits. Energy per exact product (RAPL package + `nvidia-smi`, both chips counted in both columns): 1 015 bits, $n = 1024$: 8.6 J on the GPU path against 131 J for FLINT (15×); 16 375 bits, $n = 256$: 7.9 J against 142 J (18×).

Doubling n at fixed width does not raise the ratio either (16 375 bits: 19× at $n = 256$, 17× at $n = 512$): even at $n = 512$ the conversion GEMMs ($\sim 1.9 \cdot 10^{13}$ INT8 operations) dwarf the 8 704 product GEMMs ($\sim 2.3 \cdot 10^{12}$). The ratio to FLINT falls slowly beyond ∼16 000 bits (19× → 10×) because the conversions are n^2P^2 work for P-bit entries while the products are n^3P: at $n = 128$ the conversions dominate. The way past that crossover is a subquadratic conversion (a two-level residue system), which we leave for the next version. For scale: one part in a googol (10^{-100}) is ∼333 bits, already inside the 424-bit format of the previous section.

7 Changing the problem: exact determinants and exact solutions

The ratio of the previous section falls with the bits because large numbers must enter and leave the residue system. Many exact computations produce huge numbers from small inputs: the determinant of an $n \times n$ int8 matrix has $\sim$0.72 bits per row less than Hadamard's bound, about 47 000 bits at $n = 4096$, and the rational solution of $Ax = b$ has n numerators of that size. There the conversions vanish: the input is int8, each prime returns one residue (or a vector of digits), and the only big-number step is on the host (`src/ozaki_det.cuh`, `bench/det.cu`).

LU modulo thousands of primes. For primes $p < 65\,280$ and residues X, W, with $X = X_\ell + 256X_h$ in balanced bytes and $V = 256W \bmod p$, $XW \equiv [X_\ell\, X_h][W_\ell; V_\ell] + 256\,[X_\ell\, X_h][W_h; V_h] \pmod p$: two INT8 GEMMs of depth $2K$, reduced once in the epilogue. A recursive LU (Toledo) without pivoting runs for a batch of primes at once, every $O(n^3)$ step such a product. Its 64-column leaves invert their 64×64 diagonal blocks in shared memory. A prime with a vanishing leading minor (probability $\sim n/p$) is dropped. Structured matrices, whose leading minors vanish over $\mathbb{Z}$, are preconditioned as UAL with random unit triangular U, L (Kaltofen–Saunders) whose entries in $[0, 127]$ are the same residues for every prime.

A divisor by p-adic lifting. (The divisor-then-primes hybrid is classical on CPUs [13]; FLINT uses it.) $A^{-1} \bmod p$ comes from the LU of $[A\ I]$. Dixon lifting runs on the GPU, one 16-bit and one int8 matrix–vector product per step: $x_i = A^{-1} r_i \bmod p$, $r_{i+1} = (r_i - Ax_i)/p$. Rational reconstruction of a few entries of $x = A^{-1}b$ gives d, which divides $\det A$ and for random b is almost always the largest invariant factor. Then $\det A/d$ needs only enough primes to exceed a bound on $|\det A|/d$.

A proved bound tight to one bit. Hadamard's bound leaves $\sim 0.72n$ bits of slack on a random matrix (5 900 at $n = 8192$), i.e. hundreds of primes for $\det A/d$. We replace it by a certificate in the spirit of Ogita's verified determinants [12]. FP32 LU with partial pivoting gives $PA \approx LU$. $X_L = \lfloor 2^s L^{-1} \rceil$ (unit diagonal, so exactly 2^s) and $X_U = \lfloor 2^s U^{-1} \operatorname{diag}(u) \rceil$ are integer triangular matrices, so their determinants are products of their diagonals. $B = X_L\, PA\, X_U$ is computed *exactly* (the same modular INT8 GEMMs, int64 operands reduced while packing, Garner to 128 bits), and Hadamard's inequality on B, accumulated with upward rounding, bounds $|\det A| = |\det B|/(2^{sn}\,|\det X_U|)$. Floating point only decides how close B is to $2^{2s}\operatorname{diag}(u)$, that is how tight the bound is, never whether it holds. Measured: 47 021 bits against $|\det A|$ of 47 021 bits at $n = 4096$, 98 140 against 98 139 at $n = 8192$. $\det A/d$ then needs 6 primes instead of 224 and 496.

Results. Random int8 matrices, FLINT 3.0.1 on all 24 CPU threads; every result bit-exact against FLINT. FLINT's `fmpz_mat_det` is already its accelerated path (a Dixon divisor, then primes). Its plain multimodular `fmpz_mat_det_modular` takes 347 s at $n = 2048$, 177× our all-primes LU with the same algorithm.

n	$\lvert\det A\rvert$ bits	GPU, all primes	GPU, lifting + bound	FLINT	vs FLINT
1 024	10 736	0.25 s	0.073 s	2.89 s	40×
2 048	22 485	1.96 s	0.293 s	24.9 s	85×
4 096	47 021	21.3 s	1.15 s	257 s	224×
8 192	98 139	—	8.05 s	2 861 s	355×
16 384	204 480	—	65.9 s	—	
32 768	425 348	—	594 s	—	

At $n = 8192$ FLINT ran while some GPU runs shared the CPU; the leading digits agree, and $n = 8192, 16\,384, 32\,768$ are also checked modulo four primes used nowhere else. The exact rational solution of $Ax = b$ (same lifting; all n entries back to integers by Horner in halves on 24

threads; one common denominator) is bit-exact against FLINT's `fmpz_mat_solve_dixon_den`: 0.064 s against 2.33 s at $n = 1024$ (36×), 0.145 s against 18.9 s at $n = 2048$ (130×), 1.62 s against 145 s at $n = 4096$ (90×, 192.6 Mbit of numerators), and 7.5 s at $n = 8192$ (804 Mbit). Energy per determinant at $n = 2048$, both chips counted: ∼40 J on the GPU path (process start-up included; ∼23 J for the computation alone) against ∼2 030 J for FLINT, ∼50× less. At $n = 8192$ the lifting is the cost: 13 013 steps, each reading A^{-1} (16 bits per entry) and A, at ∼100% of the read bandwidth we measure on this GPU (∼410 GB/s). That limit is physical for digit-by-digit lifting; moving past it needs lifting built on matrix products (high-order lifting, the DoublePlusOneLift of Pauderis and Storjohann [14]), the next step.

$n = 32\,768$ in 12 GB. One modular product packs $(2n + 128) \cdot 2n$ bytes three times (13 GB), and the bound holds three float and two int64 $n \times n$ matrices (30 GB). Products deeper than 2048 now run in K-chunks (a third epilogue mode, $R \mathrel{+}= XW$); the bound builds X_L, X_U in column panels (triangular solves against identity panels) kept in host memory, forms $T = X_L\, PA \bmod q$ in row panels and $B = TX_U$ in column panels as 16-bit residues on the host, and sums the rows in panels, the L_1 norms exactly in 128 bits; it runs on its own stream beside the lifting. Peak use: ∼11 GB of VRAM and 29 GB of host memory. The determinant (425 348 bits) takes 594 s: A^{-1} 15 s, 56 145 lifting steps 497 s at the bandwidth limit, $\det A/d$ 39 primes 71 s; the bound, 244 bits above $|\det A|$ at this size (FP32 LU; a larger scale 2^s does not change it), is hidden under the lifting. Each piece was checked bit-exact against FLINT at $n \le 4096$ with the chunks and panels forced small.

Large entries. With entries of L bytes stored as byte planes ($A = \sum_j 256^j A_j$), the residues come by Horner and the LU is unchanged; the lifting reads the L planes per step and keeps the residual as $L + 4$ base-256 digits per row, divided by p exactly by one lane. Lifting modulo $M = p_0 \cdots p_{S-1}$ ($S \approx L/2$ primes at once: x_i by CRT from S residues, Ax_i on byte-aligned digits, division by each p_t in turn) gains $16S$ bits per read of the L planes and cuts the lifting 1.3–2.9×. Bit-exact against FLINT: $n = 1024$ with 64-bit entries in 0.247 s against 12.4 s (50×), with 128-bit entries 0.692 s against 31.0 s (45×); $n = 2048$ with 32-bit entries 0.563 s against 46.7 s (83×), with 64-bit entries (a 137 173-bit determinant) 2.03 s against 106.9 s (53×). GPU times come from runs without FLINT beside them: CPU and GPU share one power budget, and right after 24 FLINT threads the GPU is up to 2× slower. Each step still reads all L planes, so the cost grows faster than the bits; high-order lifting [14] removes that.

8 Energy

Under these kernels the laptop holds ∼95 W and the SM clock drops to ∼1.8 GHz (2.76 GHz with only the FP64 units busy). For GEMM at $n = 4096$ (`bench/energy.cu`, power every 200 ms): native FP64 7.4 GFLOPS/W, Ozaki-II at 16 moduli 53.6 (7.2×), at 12 moduli 69.5. For a whole solve ($n = 4096$, $\kappa = 10^{20}$, 20 back-to-back GPU solves after a warm-up; CPU package power from the RAPL counters, GPU power from `nvidia-smi`):

	time	CPU package	GPU	energy per solve
`__float128` LU, 24 threads	55.0 s	106.5 W	8.4 W (idle)	6.3 kJ
dd LU on the GPU	0.238 s	28.2 W	68.3 W	**23 J**

That is ∼270× less energy per solve with both CPU and GPU counted (360× GPU alone against CPU package alone). DRAM power is not exposed on this laptop.

9 Limitations

The INT8 GEMMs are power-bound on this laptop, so a faster kernel does not translate into time; gains come from fewer operations (fewer moduli) and less data movement. The cost of the higher precisions grows roughly with the square of the number of slices, as MPFR's does at these sizes, so the ratio against MPFR does not grow without bound; it is largest against software emulation of each width. The triple-double solver is panel-bound at $n = 4096$. Kernel launches cost $\sim$7 μs under WSL2 (0.83 μs per node in a CUDA graph), which matters for the many small launches of the panel. All measurements are on one laptop model; the CPU references use the same machine's 24 threads.

10 Related work

Ozaki scheme II [1] emulates FP64 GEMM with INT8 and the CRT and reports 7.4–9.8 TFLOPS on an RTX 4090; RIKEN's GEMMul8 is its public code. Follow-up work covers SGEMM and power efficiency on an RTX 5080 [2], FP8 and FP4 variants [3, 4, 5], fused kernels reaching 81% of INT8 peak on Blackwell [6] and complex GEMM [7]. Multimodular exact integer matrix multiplication is classical (FLINT; conversions to and from a residue system written as matrix products by Doliskani, Giorgi, Lebreton and Schost [8]); Close in spirit: Ozaki-II for multiple-precision GEMM on CPU INT8 units (AMX, SVE2-i8mm) [9]; integer matrix products at various precisions on AMX and IFMA by van der Hoeven and Mezzarobba [10]; modular products over large prime fields on GPUs by Berthomieu, Graillat, Lesnoff and Mary [11]; EmuGEMM already fuses the modular reduction into the GEMM epilogue [6]. What we have not found is exact integer GEMM of 10^3–10^5-bit entries on GPU INT8 tensor cores, conversions included, measured against FLINT (to be confirmed by reading these works in full). what is new here is running it, conversions included, on INT8 tensor cores with byte-split residues. Ozaki-II mentions an extension beyond FP64 as possible; we found no implementation or measurement of a double-double or wider GEMM built this way, nor of linear solves at these condition numbers on this class of hardware. *(References to be re-checked at their source before submission.)*

11 Conclusion

On a laptop GPU whose FP64 units are 517 times slower than its INT8 tensor cores, precision becomes a software parameter: the same INT8 GEMMs yield FP64, double-double, triple-double and wider products, and an ill-conditioned dense solve that needs quad precision finishes 499$\times$ sooner than on the laptop's own CPU, with $\sim$270$\times$ less energy. Exact integer products of up to 131 063-bit entries come from the same units, 10–19$\times$ faster than FLINT. Exact determinants and exact rational solutions of int8 systems, where no big number has to enter the GPU, come 85–355$\times$ faster than FLINT. The next steps are the panel (now the bound of the triple-double solve), CUDA graphs for its launches, and out-of-core sizes beyond the GPU's 12 GB.

References

[1] K. Ozaki, Y. Uchino, T. Imamura. Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique. arXiv:2504.08009, 2025.

[2] Y. Uchino, K. Ozaki, T. Imamura. High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines. arXiv:2508.03984, 2025.

[3] Y. Uchino, K. Ozaki, T. Imamura. Double-precision matrix multiplication emulation via Ozaki-II scheme with FP8 quantization. arXiv:2603.10634, 2026.

[4] S. Hayashi, D. Mukunoki, T. Hoshino, T. Katagiri. DGEMM with Ozaki Scheme I/II on FP4 tensor cores: a base-13 E2M1 limb representation. arXiv:2608.06812, 2026.

[5] S. Hayashi, D. Mukunoki, T. Hoshino, T. Katagiri. AWE: Adaptive weight encoding for exact integer matrix products with fewer GEMMs on FP4 tensor cores. arXiv:2609.24519, 2026.

[6] D. Lu, A. Maeder, M. Luisier, A. N. Ziogas. EmuGEMM: Fused tensor core kernels for precision emulation in matrix multiplication. arXiv:2606.25453, 2026.

[7] Y. Uchino, Q. Ma, T. Imamura, K. Ozaki, P. L. Gutsche. Emulation of complex matrix multiplication based on the Chinese remainder theorem. arXiv:2512.08321, 2025.

[8] J. Doliskani, P. Giorgi, R. Lebreton, É. Schost. Simultaneous conversions with the residue number system using linear algebra. ACM Trans. Math. Softw. 44 (2018). doi:10.1145/3145573.

[9] T. Kouya. Ozaki Scheme II is fast on CPUs too: multiple-precision matrix multiplication on Intel AMX-INT8 and Arm SVE2-i8mm. arXiv:2609.27831, 2026.

[10] J. van der Hoeven, M. Mezzarobba. Integer matrix multiplication at various precisions using AMX and IFMA. Preliminary version of June 1, 2026, https://www.texmacs.org/joris/amx/amx.html.

[11] J. Berthomieu, S. Graillat, D. Lesnoff, T. Mary. Multiword matrix multiplication over large finite fields in floating-point arithmetic. arXiv:2601.07508, 2026.

[12] T. Ogita. Accurate and verified numerical computation of the matrix determinant. Int. J. Reliability and Safety 6(1/2/3) (2012) 242. doi:10.1504/IJRS.2012.044287.

[13] A. Urbańska. Towards an exact adaptive algorithm for the determinant of a rational matrix. arXiv:0706.0014, 2007.

[14] C. Pauderis, A. Storjohann. Deterministic unimodularity certification. Proc. ISSAC 2012, 281–288. doi:10.1145/2442829.2442870.